

07. Interview day

The other chapters teach you what to know. This one teaches you how to perform it in a live C++ round at a UK robotics or AI company, and gives you the oral question bank to drill in the last week. Rough time: 2 days of drills, then 20 minutes a day until the interview.

Part A — How the round actually runs

A typical robotics software round is 45–60 minutes in a shared editor (CoderPad, HackerRank, Google Docs, or a screen-shared VS Code). The usual shape:

Minutes	What happens	What they are scoring
0–5	Intro, "tell me about a project"	Can you describe a system clearly
5–35	One coding problem, usually grid/graph/array	Approach first, then clean C++, then tests
35–45	Follow-ups: complexity, edge cases, "what if the grid is 10^6 cells"	Do you understand your own code
45–55	C++ fundamentals: pointers, const, smart pointers, STL, <code>size_t</code>	Do you actually write C++
55–60	Your questions	Are you curious about their stack

Some companies replace the LeetCode-style problem with a small robotics task: "implement a PID", "write a 1D Kalman filter", "transform a point through two frames", "find the closest obstacle in a costmap". Chapter 06 covers these. The scoring is the same.

Part B — The protocol for the coding problem

Follow this every time. It is what separates "solved it" from "hire".

1. **Restate the problem in one sentence** and confirm the input format. Ask the two questions that matter: sizes ("how big can the grid be?") and edge cases ("can the start be blocked?").
2. **Say the brute force and its complexity** in one breath, then say why it is not enough.
3. **State the approach and the complexity before writing any code.** "Multi-source BFS from all rotten cells, $O(\text{rows} \times \text{cols})$ time and space." Wait for a nod.
4. **Write the signature and the tests first.** A `main()` with two `asserts`. It shows you think about correctness and gives you something to run.
5. **Write the code top-down, narrating.** Say what each block does as you type it. Silence is the most common reason a correct solution gets a weak score.
6. **Trace one small example by hand** before saying "done". Pick the example that exercises the boundary you are least sure of.
7. **Then volunteer the follow-ups:** what breaks at scale, what you would change for a real robot (fixed-size buffers, no heap allocation in the control loop, `std::array` over `std::vector`).

Things to say that cost nothing and score well:

- "I'll use `int` here because the constraints say $\leq 10^5$; if this could overflow I'd use `long long`."
- "I'll take this by `const&` to avoid the copy."
- "I'll compile this with `-Wall -Wextra`; the `size()` comparison would warn otherwise, so I'll cast."
- "This recursion is fine for 100×100 but on a 10^6 -cell costmap I'd switch to an explicit stack."

- "The priority queue has no decrease-key, so I'll push duplicates and skip stale entries."

Part C — The oral question bank

Every question below has been asked in a robotics C++ round. Answer each out loud in under 30 seconds. The short answer is here; the long explanation is in the chapter given.

Memory and objects (chapter 02)

What is the difference between a pointer and a reference? A reference is an alias that must be bound at creation and cannot be null or reseated. A pointer is a variable holding an address; it can be null, reassigned and used for arithmetic. Prefer references for parameters, pointers when "no object" is a valid state or for non-owning links.

void f(int a) { ++a; } — what does the caller see? Nothing. `a` is a copy. To modify the caller's variable, take `int&`.

What does `const` after a member function mean? The function promises not to modify the object, so it can be called on a `const` object or through a `const&`. Getters should be `const`.

Why pass `std::string` by `const&`? Passing by value copies the heap buffer. `const&` avoids the copy and documents that the function does not modify it. Pass small trivially-copyable types like `int` by value.

What is RAII? Resource Acquisition Is Initialisation: a resource is acquired in a constructor and released in the destructor, so it is released deterministically when the object goes out of scope, including when an exception unwinds the stack. `std::vector`, `std::unique_ptr`, `std::lock_guard` and `std::ofstream` all work this way.

unique_ptr vs shared_ptr? `unique_ptr` is sole ownership, zero overhead, move-only. `shared_ptr` is shared ownership with an atomic reference count, and objects die when the last owner goes. Default to `unique_ptr`; use `shared_ptr` only when lifetime is genuinely shared; use `weak_ptr` to break cycles.

Why avoid new/delete? Every `new` needs exactly one `delete` on every path, including exceptions and early returns. Containers and smart pointers do that for you. Manual `new` is a leak or double-free waiting to happen.

Stack vs heap? Stack memory is per-function, allocated by moving the stack pointer, freed automatically, fast, limited in size (~8 MB). Heap memory is allocated with `new/malloc`, lives until freed, is slower to allocate and can fragment. Never return a reference or pointer to a local variable.

Why does `while (--n >= 0)` with `size_t n` loop forever? `size_t` is unsigned. Decrementing zero wraps to the largest value, so the condition is always true. Use a signed index or loop `for (size_t i = n; i-- > 0;)`.

What is the rule of zero? If your class manages no raw resource, do not write a destructor, copy or move operations; let the compiler generate them from the members. If you must write one of the five, you probably need all five.

What is a virtual destructor for? Deleting a derived object through a base pointer is undefined behaviour unless the base destructor is virtual. Any class meant to be inherited from and deleted polymorphically needs one.

What is undefined behaviour? Give three examples. The standard places no requirement on what happens. Out-of-bounds indexing, signed integer overflow, dereferencing a dangling pointer, reading an uninitialised variable, data races.

STL (chapter 02, 05)

std::map vs std::unordered_map? `map` is a red-black tree: ordered keys, $O(\log n)$, supports `lower_bound`. `unordered_map` is a hash table: $O(1)$ average, $O(n)$ worst, no order, needs a hash for the key type. Default to `unordered_map`; use `map` when you need order or range queries.

What does `m[key]` do if the key is absent? Inserts a default-constructed value and returns a reference to it. Use `find` or `count` when you only want to look.

How do you make a min-heap with `std::priority_queue`? `std::priority_queue<T, std::vector<T>, std::greater<T>>`. The default is a max-heap.

What must a comparator for `std::sort` satisfy? Strict weak ordering: irreflexive, asymmetric, transitive. Returning `true` for equal elements (`<=`) is undefined behaviour and can crash.

What is iterator invalidation? Operations that change a container's storage make existing iterators dangling. `vector::push_back` may reallocate and invalidate everything; `erase` invalidates from the erased element onward. `std::list` and `std::map` keep other iterators valid on `erase`.

vector::reserve vs resize? `reserve` allocates capacity without changing size. `resize` changes size and value-initialises new elements. Use `reserve` before a loop of `push_back` when you know the count.

When would you use `std::array`? Fixed size known at compile time, stored inline (on the stack or inside the object), no heap allocation. Ideal for control loops and embedded code.

What is `std::string_view`? A non-owning pointer-plus-length to characters. Cheap to pass, but it dangles if the underlying string dies.

Algorithms (chapters 03, 04, 05)

Why does BFS find shortest paths in an unweighted graph? It expands nodes in order of distance from the source, so the first time a node is dequeued it is at its minimum distance. That fails with weights because a longer-hop path can be cheaper.

Dijkstra in one breath. Distances initialised to infinity except the source. Min-heap of (distance, node). Pop the smallest; if it is stale, skip; otherwise relax every edge and push improved neighbours. Works only for non-negative weights. $O((V+E) \log V)$.

Why can Dijkstra not handle negative edges? Once a node is popped it is assumed final. A later negative edge could give a shorter path to it. Use Bellman-Ford, or reweight.

Dijkstra vs A*? A* adds an admissible heuristic to the priority: $f = g + h$. With $h = 0$ it is Dijkstra. An admissible heuristic never over-estimates, so the first time the goal is popped the path is optimal; a consistent heuristic also lets you close nodes on first pop.

What heuristic for an 8-connected grid? Octile distance: $dx + dy + (\sqrt{2} - 2) \cdot \min(dx, dy)$. Manhattan is not admissible with diagonals.

BFS vs DFS: when each? BFS for shortest paths and level structure, DFS for connectivity, cycle detection, topological order, backtracking. DFS recursion depth equals the longest path; large grids need an explicit stack.

What is a topological sort and how do you detect a cycle? An order of a DAG where every edge goes forward. Kahn's algorithm repeatedly removes in-degree-zero nodes; if fewer than n nodes come out, there is a cycle.

Union-find: what does it do and how fast? Maintains disjoint sets with `find` and `union`. With path compression and union by rank each operation is amortised $O(\alpha(n))$, effectively constant.

Kth largest element: options? Sort $O(n \log n)$; min-heap of size k $O(n \log k)$; quickselect / `std::nth_element` $O(n)$ average.

How does an LRU cache get $O(1)$ for both operations? Hash map from key to a node in a doubly linked list. `get` moves the node to the front with `list::splice`; `put` evicts the back. Both are pointer operations.

Sliding window maximum in $O(n)$? A deque of indices kept decreasing by value. Pop the back while smaller than the new element, pop the front when it leaves the window; the front is the maximum.

Robotics (chapter 06)

What does the Kalman gain mean? How much to trust the measurement versus the prediction. Gain near 1: trust the sensor. Gain near 0: trust the model. It is computed from the prediction uncertainty P and the measurement noise R .

What happens if you set R very small? Q very large? Small R : the filter follows measurements almost exactly and becomes noisy. Large Q : the filter distrusts its model, P grows, and again it follows measurements. Both remove the smoothing.

Why does the integral term cause overshoot, and what is windup? The integral keeps accumulating while the error is positive, so it is still large when the error crosses zero. Windup is when the actuator saturates and the integral keeps growing anyway; fix with clamping or back-calculation.

Why quaternions instead of Euler angles? No gimbal lock, cheap composition, numerically stable renormalisation, and slerp for interpolation. Euler angles are for humans; quaternions and rotation matrices are for computation.

How do you check a 3×3 matrix is a rotation? $R^T R = I$ within tolerance and $\det R = +1$.

`T_world_base * T_base_cam * p_cam` — what is it doing? Expressing a point measured in the camera frame in the world frame by chaining transforms. Read the subscripts right to left; the inner frames cancel.

What is RANSAC and when do you use it? Repeatedly fit a model to a minimal random sample, count inliers within a threshold, keep the best, refit on its inliers. Use it when outliers would wreck least squares: plane fitting in point clouds, line detection, ICP correspondences.

How would you make this code safe for a real-time control loop? No heap allocation after initialisation (`reserve`, `std::array`, fixed pools), no locks in the loop, no exceptions on the hot path, bounded worst-case time, measure jitter.

Part D — The last week

- **Day 7-5.** Re-solve, from blank files: Number of Islands, Rotting Oranges, Network Delay Time, LRU Cache, K Closest Points, Merge Intervals. Time each at 25 minutes. Compile with `-Wall -Wextra`.
- **Day 4.** Dijkstra and A* from scratch, no references, 15 minutes each. If either takes longer, do it again the next day.
- **Day 3.** Kalman 1D and PID from scratch, 20 minutes each. Then read Part C aloud once.
- **Day 2.** One mock: pick a problem you have not seen, share your screen with a friend or record yourself, narrate for 35 minutes. Watch it back. Note the silences.
- **Day 1.** Read Part C aloud once. Prepare your three project bullets with numbers and the three questions you will ask them. Sleep.

Part E — Questions worth asking them

- What does the planning stack look like: grid, lattice, sampling-based, optimisation-based?
- How much of the codebase is C++17 or newer, and is there a style guide?
- How do you test perception and planning code: simulation, replay of logged bags, hardware-in-the-loop?
- What does a typical week look like for someone in this role?